Evan Litzer
2/19/26
COM S 418
Homework 4

3.7: Let P be a simple polygon with n vertices, which has been partitioned into monotone pieces. Prove that the sum of the number of vertices of the pieces is O(n).
Prove (pieces, Q) |V(Q)| = O(n)
1. Track the sum of vertex counts when you add a diagonal
Define S = (current pieces, Q) |V(Q)|
Only the polygon P so S0 = |V(P)| = n
- Add diagonal between two vertices u and v inside current piece Q
- This splits Q into two subpolygons Q1 and Q2 where u and v lie on the boundary of both Q1 and Q2, so u and v get counted an extra time in sum
Vertex sum: |V(Q1)| + |V(Q2)| = |V(Q)| + 2
And after adding the diagonal, S ← S + 2
2. Apply to t diagonals so sum updates for each diagonal
St = n + 2t
Where t represents each diagonal, the sum is updated by +2 due to the extra vertice count
So time bound to O(n) for now still
3. Bound the amount of diagonals t by O(n)
Diagonals are non-intersecting, so they form valid internal diagonal set of simple polygon
- Any set of non-crossing diagonals in simple n-gon has size at most n-3, as triangulation uses n - 3 diagonals and can always extend any noncrossing diagonal set to triangulation without deleting diagonals
Therefore, t <= n - 3 = O(n)
4. Bound on total vertices over all vertice
t <= n - 3 into St = n + 2t
St <= n + 2(n-3) = 3n - 6 = O(n)
Therefore, |V(Q)| = O(n) and it is proven that sum of the number of vertices of all of the pieces of the polygon is O(n)

\newpage

3.14: Given a simple polygon P with n vertices and a point p inside it, show how to compute the region inside P that is visible from p
1. Triangulate the polygon P
- Draw diagonals that are segments between polygon vertices that lie inside the polygon to decompose P into triangles
- Every simple polygon has triangulation with exactly n - 2 triangles.
- One standard pipeline is partitioned into y-monotone pieces, and then each is triangulated as monotone piece with total time O(nlogn) and linear time for all monotone pieces for second step. Then output a triangulation graph with triangles and their shared edges.
2. Find the triangle that contains P
Since p is inside P, it lies in one triangle of the triangulation
- Any point location method is used here, walking across triangles
Initialize the visible region with T0, as everything in triangle T0 is visible from p since the triangles are convex
3. Visibility propagation across triangle adjacencies
When crossing from one triangle to another through shared edge e = ab, part of space you see through the edge is contained by two rays →pa and →pb. Like a funnel structure that is used for monotone triangulation, has two bounding chains/rays

- For each crossing through shared edge, keep the portal edge ab, two directed rays from p which are named leftRay and rightRay, and the triangle on the other side
Let T have vertices (a, b, c) where ab is the shared edge
- Clip the cone using third vertex c
- If c lies inside the current cone, then parts of T are visible
- Decide which edges of T become new portals
- From triangle, only ways visibility can go outward are through edges that are not the polygon boundary yet
- For each target edge, compute the induced new cone. This is the intersection of the current cone with the rays through that edge
- Record boundary contributions
- When the edge is a polygon boundary edge, only a portion of edge lying inside the current cone is visible. This segment is the part of the final visible polygon boundary.
4. Output the visibility polygon.
- During traversal, whenever polygon boundary edge is encountered in visited triangle, add the visible subsegment which is the intersection of the edge with the current cone to the boundary list.

\newpage

- When cone narrows due to occluding vertec, add corresponding segment from p to the occlusion point
- Boundary of visible region is formed by these edge-subsegments and spokes
- Sort all boundary endpoints by polar angle around p and connect them in cyclic order to obtain visibility polygon
5. Maintain invariant that each of the portal's cone represents all of the directions from p that can pass through the portal without crossing the already blocking polygon edges
- Inside each triangle that is convex, visible set is the triangle   cone.
- Any visible point q defines segment pq crosses sequence of adjacent triangles
- Traversal reaches all triangles containing visible points. Any point outside the cone would force crossing boundary edge, so not visible
6. Triangulation is O(nlogn) using a monotone partition and triangulation. Traversal is linear in number of triangles and edges, so the total time is O(nlogn) and the storage is O(n)
4.2: Consider the casting problem in the plane: we are given polygon P and a 2-dimensional mold for it. Describe a linear time algorithm that decides whether P can be removed from the mold by a single translation.
1. Translate condition into dot products
- Polygon P has edges ei, each having outward unit normal ni that are perpendicular to the edge pointing outside P
- If P is removed by translating along direction vector d != 0, then for every edge that is contact with mold we have ni * d <= 0 because angle between d and ni is >= pi/2 is exactly ni * d <= 0
- For the top edge etop not blocked by the mold, do not require ntop * d <= 0. Need d to go out through the opening ntop * d >= 0
2. Represent directions by an angle on the unit circle
- In 2D, directions are points on unit circle where d = (cos(theta), sin(theta))
- Each constraint ni * d <= 0 cuts the circle to a closed semicircle where all directions are at least 90 degrees away from ni.
- If ni has angle i, then allowed   are   [i+2 , i+3 /2]
- Opening constraint ntop * d >= 0 is also a semicircle. [top−2 , top+2]
3. Intersect semicircle intervals in O(n) time
Since every constraint is a semicircle with interval length , intersection on the circle can be done in linear time
- Convert each constraint to angular interval [ai, bi] on [0, 2)

- Unwrap by duplicating wraparound intervals.
- If ai <= bi, then keep [ai, bi]. If ai > bi, then replace with two intervals [ai, 2) and [0, bi]
- Get the common theta by scanning

\newpage

- Initialize [L, U] = [0, 2) and for each interval constraint, intersect L ← max(L, a), U ← min(U, b)
- If ever L > U, intersection is empty
- To avoid sorting, pick reference angle theta- that is in the final intersection if it exists, so like taking any angle in the opening semicircle
- Shift all of the intervals so theta0 becomes inside the line and unwrap around theta-. Every semicircle constraint becomes normal interval on [theta0, theta0 + 2)
- Therefore linear, as O(1) work is done per edge

Full Algorithm TLDR:
Input the polygon P with edges in CCW order, and 2D mold tells you which edges form the top edge set T. Output yes if it is removable by one translation
1. Compute outward normals ni, for every polygon edge ei
2. Determine the top edge etop from the mold description
3. Build the constraints. For non-top edges ei, add constraints ni * d <= 0. Add the top constraint ntop * d >= 0
4. Convert each constraint into semicircle angular interval
5. Intersect all intervals in O(n) via the unwrap around theta0 scan
6. If the intersection is nonempty, then it is removable by one translation and output yes. Otherwise output no.

\newpage

4.7: Instead of removing the object from the mold by a single translation, we can also try to remove it by a single rotation. For simplicity, let's consider the planar variant of this version of the casting problem, and let's only look at clockwise rotations.
A. Give an example of a simple polygon P with top facet f that is not castable when we require that P should be removed from the mold by a single translation, but that is castable using rotation around a point. Also give an example of a simple polygon P with top facet f that is not castable when we require that P should be removed from the mold by a rotation, but that is castable using a single translation.
Example 1:
Why single translation does not work:
Let P be the hook polygon with vertices in CCW order
(0, 0) → (6, 0) → (6, 4) → (4, 4) → (4, 1) → (2, 1) → (2, 4) → (0, 4) → (0, 0) →
Let the top facet f be the top-left horizontal edge from (2, 4) to (0, 4)
Polygon has rectangular notch/overhang that creates second horizontal edge inside the shape
- The edge (4, 1) → (2, 1)

For every non-top edge e, direction d must make an angle that is greater than or equal to /2 with outward normal of e
- Edge e* = (4,1) → (2, 1) is horizontal and left
- The polygon is CCW, the interior lies to the left of every directed edge. For the left directed horizontal edge, the interior is below it. Therefore its outward normal points up

So for the non-top edge e*, translation condition forces d to have no upward component, so moving up would push the ceiling into the mold. But to exit through the top facet f, motion must have upward component

Any target d is pulled in opposite directions vertically by needing to go up to leave through f and needing to not go up because of the outward normal pointing up on e*
Therefore no single translation direction satisfied theorem constraints for all non-top edges, so P is not castable by translation.
Why castable using rotation around a point:
Choose pivot point r near opening, like r = (2, 4) which is endpoint of top facet. Now rotate P clockwise about r
Blocking edge e* that had outward normal straight up becomes tilted during rotation, and overhang swings out of notch instead of trying to go straight through.

\newpage

Rotation lets hook unthread from mold, right arm sweeps downward/sideways while staying inside until iut clears it, before which the rest follows through the opening.
Translation demands one fixed direction that respects all outward-normal constraints, but rotation changes the velocity direction across the object, allowing it to peel away from contact
Therefore P is a valid example, as it is not translationally castable but is rotationally castable.
Example 2:
Castable by Translation:
Let P be a thin vertical rectangle where
(0, 0) → (w, 0) → (w, H) → (0, H) → (0, 0), with H >> w and the top facet f is the top edge (w, H) → (0, H)
Take d = (0, 1) straight up. This will move the rectangle out through the top opening without needing sideways clearance
- This also matches translation feasibility as we choose direction d that respects the no-penetration constraints encoded by outward normals
Therefore P is castable by single translation
Not castable by single clockwise rotation:
With any clockwise rotation of P about any pivot point, points of P move along circular arcs centered at r.
- Unless pivot is infinite and therefore translation, some point of P must get a horizontal displacement during the motion
- In slot with width w, no room for this displacement. One of the rectangle's corners tries to move into the slot wall and causes a collision.
- Therefore, P is not castable by clockwise rotation, but is castable by translation
B. Show that the problem of finding a center of rotation that allows us
to remove P with a single rotation from its mold can be reduced to
the problem of finding a point in the common intersection of a set of
half-planes/
1. Write the instantaneous motion under clockwise rotation
- Let c be the unknown center.
- For any point q on the polygon, its instantaneous velocity under clockwise rotation is
v(q) = R-90 (q - c)
- Rotate the vector (q - c) by -90 degrees
- If q - c = (x, y), then R-90(x, y) = (y, -x)
So, v(q) = (qy - cy, -(qx - cx))
2. Translate the no collision into dot product inequality

\newpage

Take any of the edges in P e, and let ne be the outward unit normal of P along that edge. This is the edge pointing out of the polygon and into free space
- If polygon is to lift out without intersecting the mold, then at every boundary point q on that edge, the motion must not push the polygon into the mold along the outward normal

direction
- Facet blocks motion depending on the angle with outward normal condition used for translation. But the motion direction at q is the velocity v(q).
- So need ne * v(q) >= 0 for all q   e

3. Each edge gives one half-plane constraint on c

On fixed edge e, normal ne is constant and v(q) = R-90(q - c) is affine in q and c
With function g(q) = ne * R-90(q - c)
- Along line segment edge e, g(q) is linear in q so the minimum over segment occurs at an endpoint

Therefore, (q   e) g(q)   0 ↔> g(a)   0 and g(b)   0
- A and b are the endpoints of edge e

Now expand g(a) >= 0 → ne * R-90(a - c) >= 0
Since R-90(a - c) = R-90(a) - R-90(c), this equation then turns into
Ne * R-90(a) - ne * R-90(c) >= 0
↔> ne * R-90(c) <= ne * R-90(a)
- ne * R-90(c) is linear form in (cx, cy) so each inequality is half-plane in the plane of centers c

Repeat this process for endpoint b, so each edge contributes two linear inequalities, so two half-planes
So get a set H of O(n) half planes such that P is removable by single CW rotation about some center c iff the feasible region   (h   H) is nonempty

4. Reduce this to half-plane intersection
- Just run any half-plane intersection algorithm on H
- Common intersection of n half-planes can be computed in O(nlogn) time and O(n) storage

So rotation center casting problem reduces to:
- Build O(n) half-planes with 2 per edge
- Compute   H
- If empty, no valid rotation center. If nonempty, any point in the intersection is a valid center c